# Modeling Databases

**Module Outcome:**

- Draw entities and relationships before CREATE TABLE
- Pick a primary key and a foreign key so orders know their customer
- Normalise enough that AcmeHR does not store the same phone three times

## Core Concepts of ER Model

### Entity

Real world object/concepts are called **entities** in ER Model.

**Examples:** John, Emma, Apple, Google

### Attributes of an Entity

Properties of real world objects/concepts are represented as **attributes** of an entity in ER Model.

**Examples:**

1. name: John, age: 29
2. name: Emma, age: 25

### Key Attribute

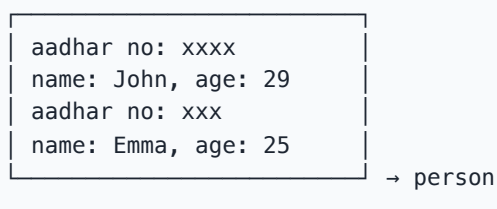
The attribute that uniquely identifies each entity is called **key attribute**.

**Example:** aadhar no: xxx, age: 29, name: John

### Entity Type

Entity Type is a collection of entities that have the same attributes (not values).

**Example:**




### Relationships

Association among the entities is called a **relationship**.

**Examples:**

- Person has a passport
- Person can have many cars
- Each student can register for many courses, and a course can have many students

## Types of Relationships

### 1. One-to-One Relationship

An entity is related to only one entity, and vice versa.

- A person can have only one passport
- Similarly, a passport belongs to one and only one person

### 2. One-to-Many Relationship

An entity is related to many other entities.

- A person can have many cars. But a car belongs to only one person

### 3. Many-to-Many Relationship

Multiple entities are related to multiple entities.

- Each student can register to multiple courses
- Similarly each course is taken by multiple students

## Cardinality Ratio

Cardinality in DBMS defines the maximum number of times an instance in one entity can relate to instances of another entity.

- one-to-one (1:1)
- one-to-Many (1:m)
- Many-to-one (m:1)
- Many-to-Many (m:n)

## Applying ER Model Concepts — E-Commerce Application

In a typical e-commerce application:

- Customer has only one cart. A cart belongs to only one customer.
- Customer can add products to cart.
- Cart contains multiple products.
- Customer can save multiple addresses in the application.

**Entity types:** Customer, Product, Cart, Address

**Relationships:**

| Relationship | Type |
|---|---|
| Customer ↔ Cart | One-to-One |
| Cart ↔ Products | Many-to-Many |
| Customer ↔ Address | One-to-Many |

**Attributes:**

| Customer | Product | Address | Cart |
|---|---|---|---|
| id (PK) | product id (PK) | id (PK) | cart id (PK) |
| name | price | pin code | total price |
| age | name | door no | |
| | brand | city | |
| | category | | |

**ER Model of E-Commerce Application:**

```
Customer --1-- has --1-- Cart
Customer --1-- has --n-- Address
Cart --m-- contains --n-- Product
```

**ER Model to Relational Database Mapping:**

- Entity Types → Tables
- Attributes → Columns
- Key Attribute → Primary Key

## Primary Keys

A minimal set of attributes (columns) in a table that uniquely identifies rows in a table.

| Table | Primary Key | Columns |
|---|---|---|
| customer | id | id, name, age |
| address | id | id, pin_code, door_no, city |
| cart | id | id, total_price |
| product | id | id, name, price, brand, category |

## Foreign Keys

**Relationship Between Customer and Address — One to Many:**

We store the primary key of a customer in the address table. This new column in the table that refers to the primary key of another table is called **Foreign Key**.

| address table |
|---|
| id (PK), pin_code, door_no, ..., customer_id (FK) |

Here, `customer_id` is the foreign key that stores `id` (primary key) of customer.

**Relationship Between Cart and Customer — One to One:**

Similar to one-to-many relationship, but we need to ensure that only one cart is associated to a customer.

| cart table |
| --- |
| id (PK), total_price, customer_id (FK, UNIQUE) |

**Relationship Between Cart and Products — Many to Many:**

We cannot store either the primary key of a product in the cart table or vice versa. To store the relationship, we use a **Junction Table** (`cart_product`).

| cart | cart_product | product |
| --- | --- | --- |
| id: 1, 2 | cart_id (FK), product_id (FK) | id, name, ... |

Properties related to a many-to-many relationship (e.g., quantity of each product in the cart) should be stored in the junction table.

**Relational Database Schema:**

| customer | address | cart | product | cart_product |
| --- | --- | --- | --- | --- |
| id, name, age | id, pincode, door_no, city, customer_id | id, total_price, customer_id | id, name, price, brand, category | id, cart_id, product_id, quantity |

## Creating a Relational Database

## Table Constraints

When creating tables, we apply constraints to enforce data integrity:

| Constraint | Purpose | Example |
| --- | --- | --- |
| `NOT NULL` | Column cannot be empty | `name VARCHAR(250) NOT NULL` |
| `UNIQUE` | No duplicate values allowed | `email VARCHAR(250) UNIQUE` |
| `DEFAULT` | Value used when none is provided | `status VARCHAR(20) DEFAULT 'active'` |
| `CHECK` | Validates a condition | `price INT CHECK (price > 0)` |
| `PRIMARY KEY` | Unique row identifier | `id INTEGER PRIMARY KEY` |
| `FOREIGN KEY` | References another table | `customer_id REFERENCES customer (id)` |

**Example:** Enforce one cart per customer using `UNIQUE` on the foreign key.

```
CREATE TABLE cart (
    id INT NOT NULL PRIMARY KEY AUTO_INCREMENT,
    customer_id INTEGER NOT NULL UNIQUE,
    total_price INTEGER,
    FOREIGN KEY(customer_id) REFERENCES customer(id) ON DELETE CASCADE
);
```

`UNIQUE` on `customer_id` ensures the one-to-one relationship between customer and cart.

## Primary Key Syntax

```
CREATE TABLE table_name (
    C1 t1 NOT NULL PRIMARY KEY,
    ...
    Cn tn
);
```

## Foreign Key Syntax

```
CREATE TABLE table2 (
    C1 t1 NOT NULL PRIMARY KEY,
    C2 t2,
    FOREIGN KEY (C2) REFERENCES table1 (C3)
    ON DELETE CASCADE
);
```

`FOREIGN KEY (C2) REFERENCES table1 (C3)` ensures that foreign key can only contain values that are in the referenced primary key.

**ON DELETE CASCADE:** If a row in table1 is deleted, then all its related rows in table2 will also be deleted.

> **Note:** In MySQL, foreign key constraints are enforced automatically when using the **InnoDB** storage engine (the default). Ensure referenced tables use InnoDB: `ENGINE=InnoDB`.

## Creating Tables

```
CREATE TABLE customer (
    id INT NOT NULL PRIMARY KEY AUTO_INCREMENT,
    name VARCHAR(250),
    age INT
);

CREATE TABLE product (
    id INT NOT NULL PRIMARY KEY AUTO_INCREMENT,
    name VARCHAR(250),
    price INT,
    brand VARCHAR(250),
    category VARCHAR(250)
);

CREATE TABLE address (
    id INT NOT NULL PRIMARY KEY AUTO_INCREMENT,
    pin_code INTEGER,
    door_no VARCHAR(250),
    city VARCHAR(250),
    customer_id INTEGER,
    FOREIGN KEY(customer_id) REFERENCES Customer(id) ON DELETE CASCADE
);

CREATE TABLE cart (
    id INT NOT NULL PRIMARY KEY AUTO_INCREMENT,
    customer_id INTEGER NOT NULL UNIQUE,
```

```
    total_price INTEGER,
    FOREIGN KEY(customer_id) REFERENCES customer(id) ON DELETE CASCADE
);

CREATE TABLE cart_product (
    id INT NOT NULL PRIMARY KEY AUTO_INCREMENT,
    cart_id INTEGER,
    product_id INTEGER,
    quantity INTEGER,
    FOREIGN KEY(cart_id) REFERENCES cart(id) ON DELETE CASCADE,
    FOREIGN KEY(product_id) REFERENCES product(id) ON DELETE CASCADE
);
```